Chapter 3 Exercises

State width, signedness, rounding mode, and iteration number. Use binary32 for unspecified floating-point questions.

CH03-Q01 Addition and subtraction

Use eight-bit two's complement.

1. Find bits, signed value, and overflow for `0x7f + 0x01`.
2. Do the same for `0x80 + 0xff`.
3. Find bits and overflow for `0x80 - 0x01`.
4. Express signed overflow using carries at the top bit.

CH03-Q02 Sequential multiplication

Use four-bit unsigned shift-and-add with four-bit A and Q. Each addition temporarily retains carry C, so C:A=A+M. Shift the nine-bit `(C,A,Q)` right, moving C into A's MSB and discarding processed Q0. Initially A=0000,M=1101=13,Q=1011=11.

1. List old Q0, C:A, and A/Q after every shift.
2. Decode the final eight-bit product and check decimal multiplication.
3. Explain the general n-bit correctness using $\sum q_k(M2^k)$. A four-round trace alone is insufficient.
4. For n=32, add and shift each cost one cycle, setup three, and finalization two. Find serial latency. If an implementation accepts a new operation every four cycles, find throughput and state the extra hardware/context needed.

CH03-Q03 Restoring division

Use five-bit remainder A, four-bit quotient Q, divisor $M = 00101_2$, and initial dividend $Q = 1101_2$. Each iteration shifts combined state left, subtracts M, and either retains the nonnegative result with quotient bit 1 or restores it with bit 0.

1. Trace four rounds.
2. Find quotient and remainder.
3. Verify $D = QM + R$ and the remainder bound.
4. Analyze signed 32-bit `0x80000000 / 0xffffffff`.
5. Independently trace 11/3, listing prefix P_k, trial T, generated quotient q_k, and remainder A_k at every round. Distinguish q_k from the entire hardware Q. Prove the update using $P_k = q_k M + A_k$ and $0 \le A_k < M$.
6. Under truncation toward zero, find quotient and remainder for −11/3 and −11/(−3). What changes if −11/3 uses floor division? Check both the identity and remainder bound.

CH03-Q04 Binary32 encoding

1. Encode −12.5 in hexadecimal.
2. Decode `0x3f000000`.
3. Give fields and powers of two for the smallest positive normal and subnormal.
4. Classify `0x7f800000` and `0x7fc00001`.

CH03-Q05 Nearest-even

Compute binary32 $1 + 2^{-24}$ using nearest-even.

1. Give the neighboring candidates.
2. Identify the tie and lowest retained bit.
3. State the result and status.
4. Explain how retaining greater intermediate precision can affect a later subtraction by 1.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CH03-Q06 Floating-point multiplication

Let binary32 $a = 1.5$ and $b = 2^{-126}$.

1. Give significands and unbiased exponents.
2. Find the exact significand product and exponent.
3. How many normalization left shifts are needed?
4. Why is adding stored exponent fields insufficient?

CH03-Q07 Nonassociativity

Use binary32 nearest-even with $x = 2^{24}, y = 1$.

1. Compute $(x + y) - x$ in order.
2. Compute $(x - x) + y$.
3. Locate the rounding that causes the difference.
4. Compare $(1 + 2^{-24}) - 1$ when an x87-style wider intermediate retains the sum exactly and when every operation rounds to binary32.

CH03-Q08 Arithmetic timing

A sequential multiplier takes 32 one-cycle iterations, three setup cycles, and two finalization cycles. A radix-4 model takes 16 iterations of 1.5 cycles each and six combined setup/finalization cycles. Both systems explicitly have enough replicated units or interleaved contexts to accept a new operation every eight cycles. A single unpipelined iterative unit does not naturally provide this capability.

1. Find each single-operation latency.
2. Find each steady-state throughput.
3. When does latency matter, and when does throughput matter?
4. Why do 16 iterations not imply twice the speed? What additional costs would a divider need?

Lin Cing

CH03-Q09 Rounding and exceptions

Use binary32 round-to-nearest-even. Interpret values below the minimum positive subnormal by their exact mathematical value.

1. Give G/R/S for `1.0+2^-24`, its result, and inexact status.
2. Give G/R/S and result for `1.0+3*2^-25`.
3. Round 2^{-151} and 2^{-150}. For the latter, identify both candidates, the selected candidate, and flags.
4. Give default results and exceptions for maximum finite binary32 plus itself and `0*infinity`.

CH03-Q10 Single and separate rounding

Let binary32 $a = b = 1 + 2^{-23}$ and $c = -(1 + 2^{-22})$.

1. Give a and c in hexadecimal.
2. Execute multiply then add c, retaining the exact product, rounded product, and final bits.
3. Compute the exact result and bits of a fused RNE operation with one rounding.
4. Use $x = 2^{24}$ to compare `(x+1)-x` and `(x-x)+1`. Identify the changed rounding point and why exact associativity does not transfer to per-operation binary32 arithmetic.

CH03-Q11 Alignment, cancellation, and rounding carry

Use binary p=4 including the leading 1, RNE, and an unbounded exponent range.
(1) For a=1,b=9/128, give the aligned bits, L/G/R/S, adjacent candidates, decision, and absolute error. What wrong result follows if sticky is ignored?
(2) Compute 0.75−0.6875, listing the significand and exponent after subtraction and each left shift. Determine inexact.
(3) Round exact 1.1111_2 and -1.0011_2. Give candidates and parity, identify renormalization, and explain the increment direction for a negative result.

CH03-Q12 Subnormals and flags

Use binary32, RNE, gradual underflow, disabled traps, tininess after rounding, and initially clear flags. Do not use FTZ or treat subnormal inputs as zero. Let $\delta = 2^{-149}$.

1. For $2^{-126}/2$, $\delta/2$, and $(3\delta)/2$, give exact values, adjacent candidates when rounding is needed, output bits, underflow, and inexact.
2. Compute smallest normal minus largest subnormal. Give exact value and bits. Does this operation set underflow?
3. Use the fraction integer F to derive subnormal spacing and determine whether a gap appears at the smallest normal.

CH03-Q13 Booth bits, widths and correctness

Use the lesson's radix-2 model: $n = 4$, five-bit A, M, four-bit Q, one history bit, and add/subtract before each concatenated arithmetic shift.

1. For $M = -8, Q = -3$, trace the initial state, every old pair, and all bits before and after each shift. Decode the final eight-bit product.
2. If A, M are incorrectly restricted to four bits, locate the first error and trace the incorrect final product. Why is the fifth bit necessary?
3. List radix-2 digits and arithmetic count for multiplier `0101`, comparing with unsigned shift-add. Do four rounds directly imply four clock cycles?
4. In six-bit radix-4, compute $M = -5, Q = -13$. Give three overlapping groups, digits, twelve-bit partial products and their sum. Derive the table using $c_j = d_{2j} + 2d_{2j+1}$.

CH03-Q14 Reduce rows while preserving their value

1. Generate four eight-bit partial products for unsigned four-bit M=11,Q=15. Compress P_0,P_1,P_2, then S_1,C_1,P_3. Give both S and already-shifted C, their bit patterns and preserved decimal total at each level.
2. Perform the final ordinary addition, listing every carry-in and carry-out from low to high. What final value results if both erroneous reduction levels omit the majority-row left shift?
3. Prove X+Y+Z=S+C for arbitrary W-bit inputs from the one-column full-adder identity. For W=4,X=Y=Z=15, distinguish the exact result from the low-four-bit result and explain required widths.
4. How many levels reduce nine rows to two under the three-row grouping model? With t_PP=40 ps,t_CSA=60 ps,t_CPA=180 ps and extra t_wire=20 ps, evaluate the delay bound. Does it establish superiority over another multiplier?
5. A three-stage pipeline has period T=250 ps and no stalls. Give acceptance/completion edges and times for five consecutive independent transactions, first-result latency, initiation interval and final completion. Which assumption fails if transaction two needs transaction one's result?

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study